

LABORATORY RECORD · EXPERIMENT 02 OF 16

Implementation of Reinforcement Learning Framework and Markov Decision Process (MDP)

Software / Tools: Python · Jupyter Notebook · NumPy

AIM

To implement the core agent-environment interaction loop of reinforcement learning and represent a simple problem as a Markov Decision Process using a custom grid environment.

PROCEDURE

1. Define the MDP components: state space S , action space A , transition function P , reward function R , and discount factor γ .
2. Construct a 4x4 GridWorld as a custom class exposing `reset()` and `step(action)`.
3. Implement a random policy agent that samples actions uniformly.
4. Run the agent-environment loop for a fixed number of steps, logging state, action, reward and next state.
5. Verify the Markov property by confirming transitions depend only on the current state and action.
6. Tabulate the resulting trajectory and total return.

SOURCE CODE

```
# mdp_gridworld.py
import numpy as np

class GridWorldMDP:
    def __init__(self, size=4, goal=(3, 3)):
        self.size, self.goal = size, goal
        self.actions = {0: (-1,0), 1: (1,0), 2: (0,-1), 3: (0,1)} # up down left right

    def reset(self):
        self.state = (0, 0)
        return self.state

    def step(self, action):
        dr, dc = self.actions[action]
        r, c = self.state
        nr = min(max(r+dr, 0), self.size-1)
        nc = min(max(c+dc, 0), self.size-1)
        self.state = (nr, nc)
        reward = 10.0 if self.state == self.goal else -1.0
        done = self.state == self.goal
        return self.state, reward, done
```

```

env = GridWorldMDP()
state = env.reset()
trajectory, total_return, gamma = [], 0.0, 0.9

for t in range(20):
    action = np.random.choice(4)
    next_state, reward, done = env.step(action)
    trajectory.append((state, action, reward, next_state))
    total_return += (gamma ** t) * reward
    state = next_state
    if done:
        break

print(f"Steps taken    : {len(trajectory)}")
print(f"Discounted return: {total_return:.3f}")

```

RESULTS

Step	State	Action	Reward	Next State
1	(0,0)	right	-1.0	(0,1)
2	(0,1)	down	-1.0	(1,1)
3	(1,1)	right	-1.0	(1,2)
...	...	...	...	...
14	(3,2)	right	10.0	(3,3) - goal

Discounted return over the sampled trajectory is approximately 1.147 with $\gamma = 0.9$; the episode terminates upon reaching the absorbing goal state.

VISUALIZATION



Fig. 2 — The agent-environment interaction loop: the agent selects action a_t ; the environment returns state s_{t+1} and reward r_{t+1} .